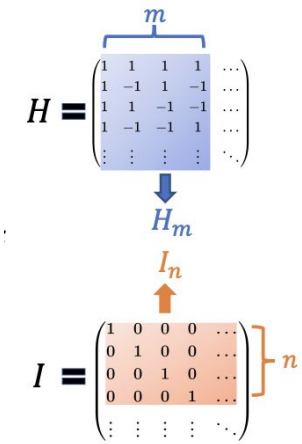
ZerO Initialization

Простое объяснение. Логинов Никита

Коротко о чем

Мы хотим инициализировать веса нашей модели не случайно, а используя нули и единицы. В этом нам помогут единичные матрицы (матрица I на рисунке) и матрицы Адамара (матрица H на рисунке).



Зачем? Хотим добиться вычислительной эффективности, при этом не теряя в качестве

А в чем минусы случайной инициализации?

Почти всегда мы инициализируем наши веса случайно. Самое важное здесь - правильно подобрать дисперсию распределения.

Если дисперсия будет слишком большой - будет чрезмерное усиление градиентов -> взрыв градиентов. **Неприятно**

Если будет слишком маленькой - можем так и не сойтись к оптимуму, тк градиент затухнет. **Обидно**

Статья как раз про то, как не страдать с изначальной инициализацией весов и при этом не просесть в качестве.

В таком случае, почему не можем инициализировать константами?

В таком случае одинаковые веса считают одинаковые значения, получают одинаковые градиенты, одинаково обновляются. Это называется **проблемой симметрии**, в таком случае наша сетка не будет такой экспрессивной, как могла бы быть при случайной инициализации весов.



Окей, а почему не можем инициализировать просто единичными матрицами, зачем нам еще какие-то матрицы Адамара?

Можем!!! Другие авторы в своих статьях использовали только единичные матрицы, но делали они это для сеток с одинаковой размерностью на всех слоях.

Как мы понимаем, на практике такого не будет никогда. Взять тот же resnet, размерность его слоев различается.

Проблема использования только единичных матриц

Если мы будем использовать только единичные матрицы* в сетках, где размерности внутри сети постоянно меняется, то столкнемся с проблемой **training degeneracy**. Какой бы большой ни была внутренняя размерность сети, активации будут находиться в подпространстве, ограниченном сверху входной размерностью.

Простыми словами: сеть стала шире, но не может воспользоваться своей увеличенной размерностью и быть более экспрессивной. Пространство остается низкоранговым.

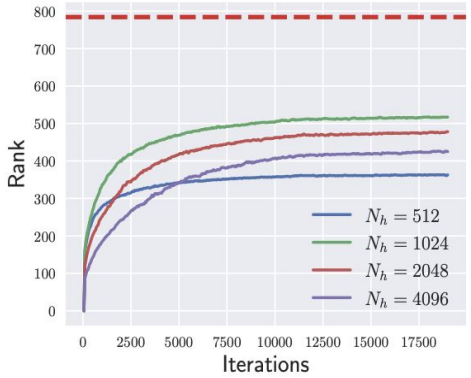
**(или partial identity matrix при увеличении/уменьшении размерности)*

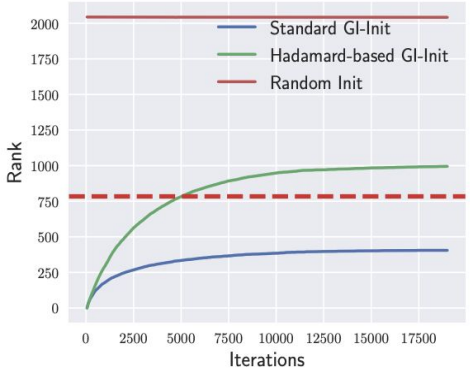
Как исправить это?

В слоях, где мы увеличиваем размерность, мы будем использовать матрицу Адамара для инициализации весов. В слоях, где при переходе размерность не меняется, мы будем использовать единичные матрицы. Если размерность уменьшается, будем использовать partial identity matrix.

Матрицы Адамара распределяют вход между разными направлениями, в том числе и новыми. Таким образом мы не будем ограничены входной размерностью.

Пример на картинке





Отслеживаем изменение ранга матрицы W2(произвольная матрица весов сетки) во время обучения. Красная пунктирная линия обозначает теоретическое ограничение ранга W2, равное размерности входа n_x=784

Слева: Независимо от того, насколько велико n_h(hidden dim), ранг W2 остаётся в пределах ограничения на протяжении всего обучения.

Справа: Применение преобразования Адамара снимает ограничение ранга

Важный нюанс метода ZerO

Когда я сказал, что будем использовать только нолики и единички, я вас немного обманул.

В статье используют нормированные матрицы Адемара, чтобы избежать взрыва градиентов и получить более стабильное обучение.

Это не ломает смысл статьи, тк ребята просто хотят показать, что можно инициализироваться константами и при этом не пострадать.



Как инициализируем свертки в ZerO

Авторы помещают единичное или преобразование Адамара в центральную позицию ядра, а остальные элементы устанавливают в ноль.

Например, для ядра 3×3 ненулевым является его центр.

Если число выходных каналов не увеличивается, центр ядра выполняет единичное отображение каналов.

Последнюю свёртку residual блока инициализируем нулём. В начале residual блок ведёт себя примерно как единичная матрица.

Результаты из статьи

Dataset	**Model**	**Initialization**	**Test Error** (mean ± std)
CIFAR-10	ResNet-18	**ZerO Init**	5.13 ± 0.08
		Kaiming Init	5.15 ± 0.13
		Xavier Init	5.23 ± 0.16
ImageNet	ResNet-50	**ZerO Init**	23.43 ± 0.04
		Kaiming Init	23.46 ± 0.07
		Xavier Init	23.65 ± 0.11

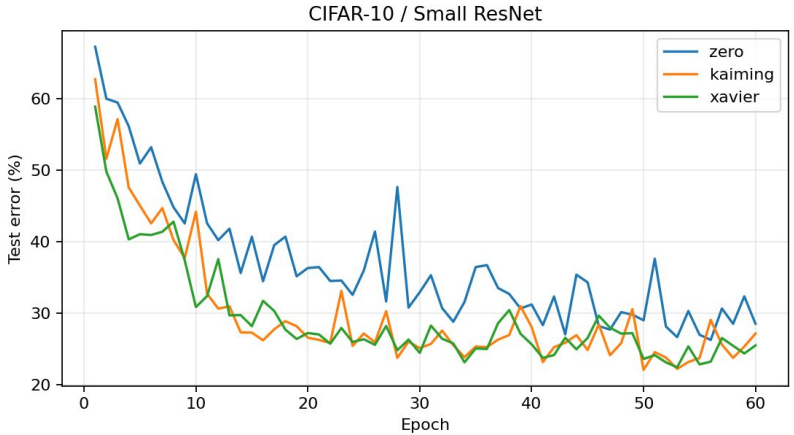
Каждую модель прогнали 10 раз с разными random seed, чтобы получить более точную оценку, показать дисперсию. Как видно, ZerO обладает меньшей дисперсией среди всех методов, что в принципе неудивительно, поскольку в методе используем константную инициализацию.

Можем увидеть, что ZerO не ухудшает качество итоговой модели.

Результаты моего прогона

Я тоже взял CIFAR10+resnet, но более легковесный, чтобы обучение не растянулось на весь семестр (учил в коллабе на T4). Обучал 60 эпох (в статье обучали 200) с разными вариантами инициализаций. Подмечу - графиков обучения в статье не прилагается, сравнить не с чем.

Мои результаты показывают, что обучение при ZerO проходило менее стабильно, но в итоге пришло к сравнительно схожей test error (ZerO - 28%, Xavier - 25.5%, kaiming - 27%). Думаю сыграло роль то, что воспроизводил результаты не в полной точности (как минимум обучал 60 эпох, а не 200) + при ZerO методе матрицы изначально низкоранговые, что сказывается на выразительности (но об этом далее)



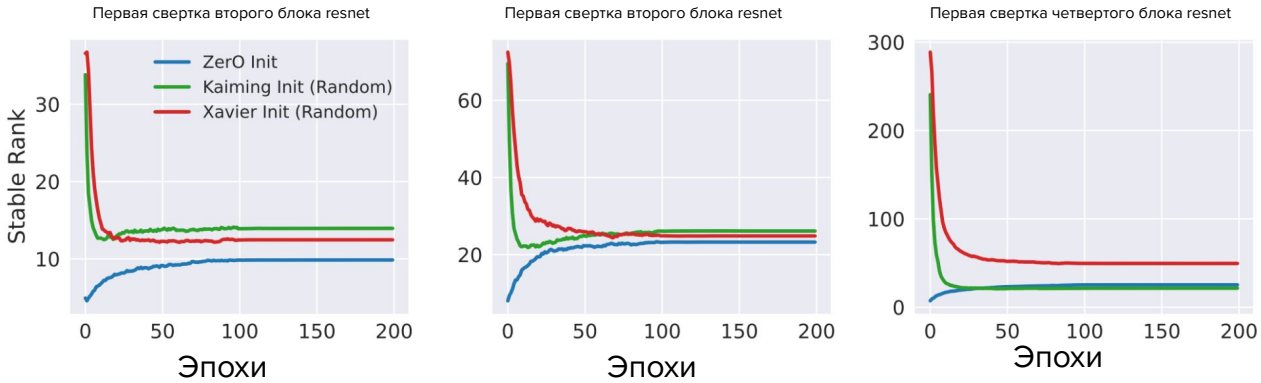
Low-Rank learning trajectory(результаты статьи)

При ZerO инициализации мы можем увидеть, что матрицы в начале обучения начинают как простые, низкоранговые, а потом увеличивают ранг по мере необходимости.

При случайной инициализации матрицы с первой эпохи имеют более высокий ранг.

Утверждается, что ZerO ведет к более простым, компактным решениям, не заставляя сеть исследовать более сложное пространство.

Эффективность заключается в том, что если много весов близки к нулю, их можно удалить. Тогда остаётся меньше активных параметров и потенциально меньше умножений/сложений при инференсе. При ZerO можно удалить больше околонулевых/нулевых весов без существенной потери качества.



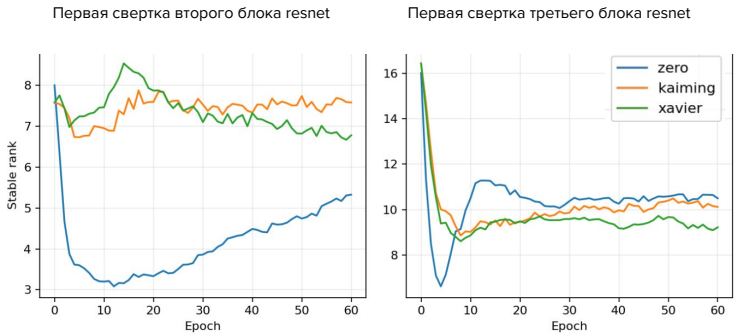
Мои результаты изучения **low-rank learning trajectory**

Мои результаты частично подтвердили статью: на свертке второго блока явно видно, что при ZerO инициализации матрица имеет меньший stable ранг*. Однако для третьего блока такой тенденции заметить нельзя: даже при ZerO инициализации мы сходимся к тому же (или даже большему) рангу, в сравнении с другими методами инициализации весов.

Четвертого блока в моей сетке не было, поэтому сравнить эту часть не получилось.

Вывод: добиться матриц меньшего ранга возможно, но такой результат будет негарантирован

*такой большой разрыв связан с тем, что обучал мало эпох. Если увеличить количество, то, как и в результатах статьи, разрыв будет меньше



Вывод

Авторы статьи показали, что необязательно инициализироваться случайными весами для того, чтобы добиться сравнимого качества.

В своих результатах я заметил, что на протяжении всего обучения ошибка при ZerO инициализации была больше на тестовой выборке, пусть и на небольшой процент. Думаю, это как раз связано с тем, что матрицы в ходе обучении имеют низкий ранг и повышают его с течением обучения, тогда ошибка становится сравнимой с другими методами.

Что касается low-rank learning trajectory: в своих результатах, я не увидел большой разницы в рангах к концу обучения, от этого не вижу значительной эффективности метода. Думаю, на это повлияло то, что я обучаю значительно более легкую стеку на меньшем количестве эпох.

Если опираться на результаты статьи, то можно ожидать выигрышь при прунинге вашей сетки, тк ZerO часто ведет к более разреженным и низкоранговым решениям

